

PRAKTIKUM SISTEM OPERASI

MODUL XI
MEMORI XINU



Oleh:

Zaenarif Putra 'Ainurdin

2311104049

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM

2026

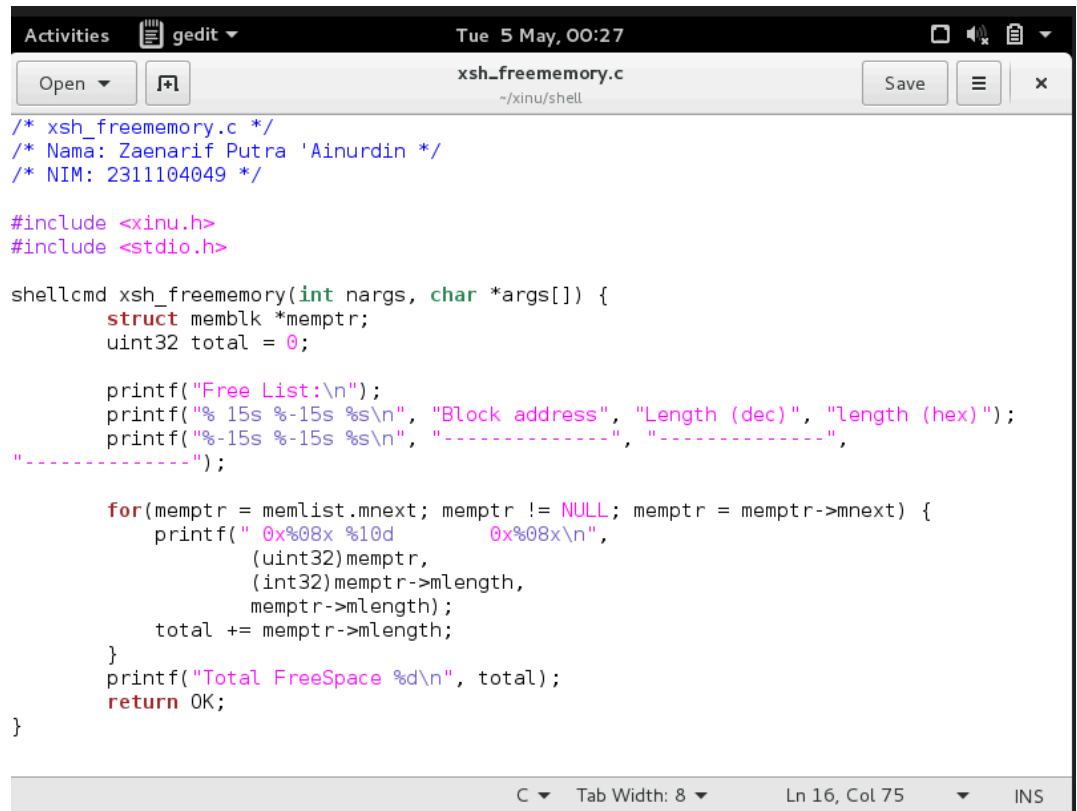
JURNAL

1. Soal 1

Buatlah perintah baru bernama freememory yang memiliki dua fungsi berikut:

- Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
- Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

- Syntax xsh_freememory.c:



```
/* xsh_freememory.c */
/* Nama: Zaenarif Putra 'Ainurdin */
/* NIM: 2311104049 */

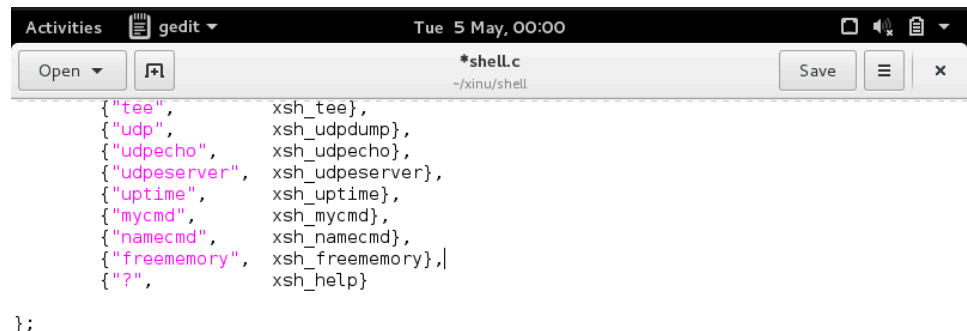
#include <xinu.h>
#include <stdio.h>

shellcmd xsh_freememory(int nargs, char *args[]) {
    struct memblk *memptr;
    uint32 total = 0;

    printf("Free List:\n");
    printf("% 15s % 15s %s\n", "Block address", "Length (dec)", "length (hex)");
    printf("%-15s %-15s %s\n", "-----", "-----",
    "-----");

    for(memptr = memlist.mnext; memptr != NULL; memptr = memptr->mnext) {
        printf(" 0x%08x %10d 0x%08x\n",
            (uint32)memptr,
            (int32)memptr->mlength,
            memptr->mlength);
        total += memptr->mlength;
    }
    printf("Total FreeSpace %d\n", total);
    return OK;
}
```

- Tambahkan juga di shell.c:



```
{ "tee", xsh_tee},
{ "udp", xsh_udpdump},
{ "udpecho", xsh_udpecho},
{ "udpserver", xsh_udpserver},
{ "uptime", xsh_uptime},
{ "mycmd", xsh_mycmd},
{ "namecmd", xsh_namecmd},
{ "freememory", xsh_freememory},
{ "?", xsh_help}

};
```

- Tambahkan juga di shprototypes.h



```

/* in file xsh_argecho.c */
extern shellcmd xsh_argecho (int32, char *[]);

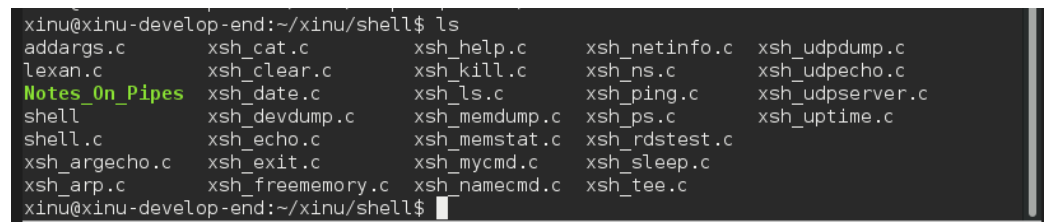
/* in file xsh_namecmd.c */
extern shellcmd xsh_namecmd (int32, char *[]);

/* in file xsh_arp.c */
extern shellcmd xsh_arp (int32, char *[]);

/* in file xsh_freememory.c */
extern shellcmd xsh_freememory (int32, char *[]);

```

- Ls directory freememory:

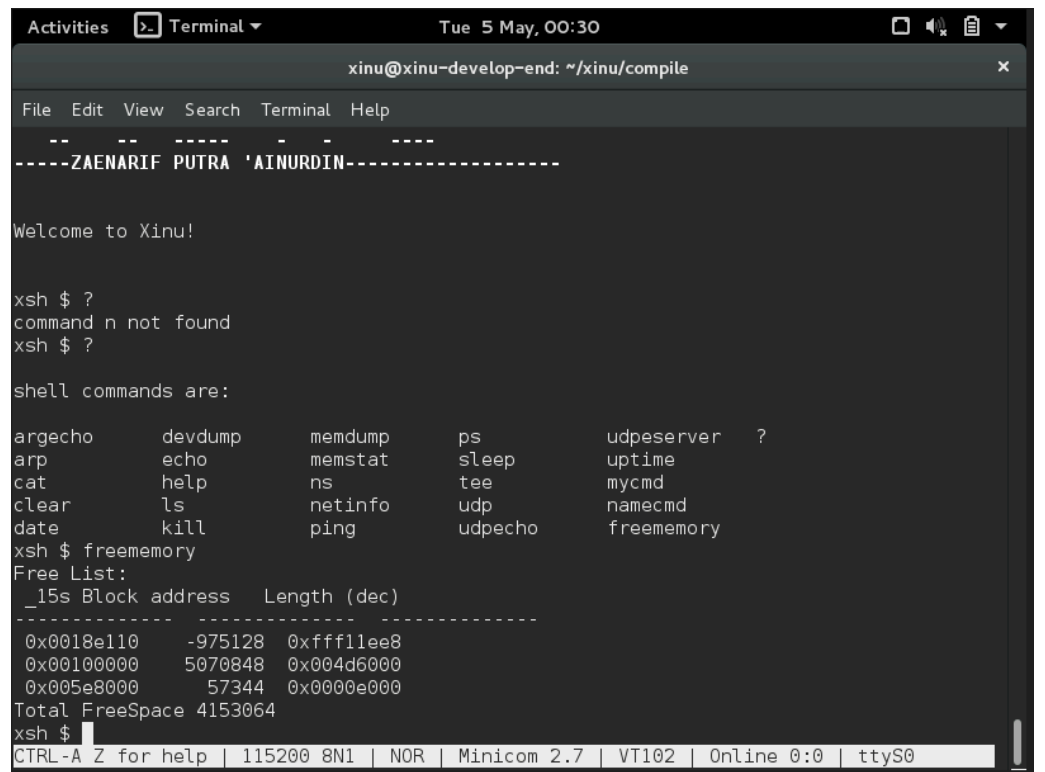


```

xinu@xinu-develop-end:~/xinu/shell$ ls
addargs.c      xsh_cat.c      xsh_help.c      xsh_netinfo.c  xsh_udpdump.c
lexan.c        xsh_clear.c    xsh_kill.c      xsh_ns.c       xsh_udpecho.c
Notes_On_Pipes xsh_date.c     xsh_ls.c        xsh_ping.c     xsh_udpserver.c
shell          xsh_devdump.c  xsh_memdump.c   xsh_ps.c       xsh_uptime.c
shell.c        xsh_echo.c     xsh_memstat.c   xsh_rdstest.c
xsh_argecho.c  xsh_exit.c     xsh_mycmd.c     xsh_sleep.c
xsh_arp.c      xsh_freememory.c xsh_namecmd.c   xsh_tee.c
xinu@xinu-develop-end:~/xinu/shell$

```

- Output yang dihasilkan:



```

xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help
-- -- -----
-----ZAENARIF PUTRA 'AINURDIN-----

Welcome to Xinu!

xsh $ ?
command n not found
xsh $ ?

shell commands are:

argecho    devdump      memdump       ps            udpserver    ?
arp         echo         memstat       sleep         uptime
cat         help         ns            tee           mycmd
clear      ls           netinfo       udp           namecmd
date       kill         ping          udpecho       freememory

xsh $ freememory
Free List:
_15s Block address  Length (dec)
-----
0x0018e110  -975128  0xffff11ee8
0x00100000  5070848  0x004d6000
0x005e8000  57344   0x0000e000
Total FreeSpace 4153064
xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS0

```

2. Soal 2

Jawablah pertanyaan berikut:

a. Mengapa Xinu memisahkan data segment dan BSS segment?

Xinu (dan sistem operasi berbasis Unix pada umumnya) memisahkan data segment dan BSS segment karena alasan efisiensi penyimpanan pada file executable dan kecepatan loading:

- Data segment berisi variabel global yang sudah diinisialisasi dengan nilai tertentu (misalnya `int x = 5;`). Nilai-nilai ini harus disimpan secara eksplisit di dalam file executable agar tersedia saat program dimuat ke memori.
- BSS segment (Block Started by Symbol) berisi variabel global yang tidak diinisialisasi atau diinisialisasi dengan nilai nol (misalnya `int y;`). Karena nilainya nol semua, tidak perlu menyimpan data aktualnya di dalam file executable. Cukup menyimpan ukuran total BSS, lalu OS akan mengisi area tersebut dengan nol saat loading.
- Pemisahan ini menghemat ukuran file executable secara signifikan: jika ada array besar yang tidak diinisialisasi seperti `char buf[1048576];`, file executable tetap kecil karena BSS hanya mencatat ukurannya, bukan datanya.
- Selain itu, OS dapat menggunakan teknik zero-fill-on-demand untuk BSS, sehingga alokasi memori fisik juga lebih efisien.

b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

Free space pada Xinu bersifat dinamis dan terus berubah seiring aktivitas proses:

- Saat proses baru dibuat (`create`): Xinu memanggil `getstk()` untuk mengalokasikan stack bagi proses tersebut dari alamat memori tertinggi ke bawah. Ukuran free space berkurang sebesar ukuran stack yang dialokasikan.
- Saat proses meminta heap (`getmem()`): Xinu mengalokasikan blok dari alamat free space terendah ke atas. Ukuran free space berkurang lagi.
- Saat proses terminasi (`freestk()`): Stack dikembalikan ke free list secara otomatis, ukuran free space bertambah kembali.
- Saat proses membebaskan heap (`freemem()`): Heap dikembalikan secara manual dan eksplisit oleh proses. Jika proses lupa memanggil `freemem()`, terjadi memory leak dan free space tidak pulih.
- Fragmentasi: Alokasi dan dealokasi yang berulang dapat menyebabkan free space terpecah menjadi banyak blok kecil yang tidak bersebelahan (fragmentasi), sehingga meskipun total free space cukup, blok besar yang kontinu tidak tersedia.

- c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?

Heap dikelola secara manual. Sehingga programmer perlu bertanggung jawab penuh atas alokasi dan dealokasi. Kesalahan seperti double free, use-after-free, atau lupa free mem menyebabkan korupsi memori yang sulit didebug.

- d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?

Xinu menggunakan singly linked list untuk mengelola daftar free memory block karena beberapa alasan:

- Ukuran dan posisi free block tidak tetap: Setelah berbagai alokasi dan dealokasi, free block tersebar di berbagai lokasi dengan ukuran berbeda-beda. Array berukuran tetap tidak cocok karena jumlah free block berubah dinamis.
- Efisiensi memori untuk metadata: Linked list menyimpan informasi (ukuran blok dan pointer ke blok berikutnya) di dalam free block itu sendiri (header blok). Tidak diperlukan struktur data eksternal tambahan.
- Kemudahan penggabungan (coalescing): Saat suatu blok dibebaskan dan bersebelahan dengan blok bebas lain, linked list memungkinkan penggabungan blok tersebut menjadi satu blok besar dengan mudah.
- Traversal fleksibel: Xinu dapat menelusuri seluruh free block untuk mencari blok dengan ukuran yang sesuai (first-fit atau best-fit). Linked list mendukung traversal sequential dengan overhead rendah.
- Overhead rendah: Untuk sistem embedded seperti Xinu, struktur linked list sederhana (hanya menyimpan mlength dan mnext) lebih efisien dibandingkan struktur data kompleks seperti tree atau hash table.

- e. Apa tantangan utama dalam penggunaan heap di Xinu?

Terdapat tiga tantangan utama dalam penggunaan heap di Xinu:

- Dealokasi Manual (Memory Leak)
Berbeda dengan stack yang secara otomatis dibebaskan saat proses terminasi, heap harus dibebaskan secara manual menggunakan freemem(). Jika proses terminasi tanpa memanggil freemem(), memori heap tersebut tidak dikembalikan ke free list dan menjadi memory leak permanen sampai sistem di-reboot.
- Fragmentasi Memori
Alokasi dan dealokasi heap yang tidak beraturan menyebabkan fragmentasi eksternal: free space terpecah menjadi banyak blok kecil yang terpisah. Meskipun total free space mencukupi, permintaan alokasi blok besar yang kontinu bisa gagal karena tidak ada satu blok pun yang cukup besar. Xinu tidak memiliki mekanisme kompaksi memori (memory compaction) karena pointer langsung ke alamat fisik.

- **Kehabisan Free Space**

Pada sistem embedded dengan memori terbatas, permintaan heap dapat melebihi free space yang tersedia. Xinu tidak memiliki mekanisme virtual memory atau swap space, sehingga jika free space habis, `getmem()` akan gagal dan proses tidak dapat melanjutkan eksekusi. Selain itu, stack dan heap tumbuh ke arah yang berlawanan sehingga collision keduanya juga menjadi risiko.